

MLH PE Fellowship Notes

Maninder (Kaurman) Kaur

July 16, 2026

Week 2: Linux Fundamentals

Linux is an OS specifically better for open applications, more versatile, and overall better privacy options. All terminal functions run on Linux commands, so let's learn that.

Scenario: System is Slow. To see why it's slow, they could use the **top** command to show summary info of list of processes or threads running on the Kernel.

Next, check the Disk IO using **iostat** that checks the input/output to see CPU utilization, number of transfer/sec, bulk reads/writes and CPU metrics.

Then, maybe they use **sar** that sees overall system performance. We could monitor things like CPU usage, Memory utilization, I/O device consumption, battery performance, etc.

Folders in Linux

/	primary directory
/bin	essential executable programs
/boot	files needed to boot system, such as kernal, boot config files, etc.
/dev	devices nodes, used to interact with hardware
/etc	system-wide config files
/home	user home directories
/lib	libraries required for /bin and /sbin
/lib64	64-bit libraries
/media	mount points for removable CDs, DVDs, etc.
/mnt	temporarily mounted filesystems
/opt	optional application software packages
/proc	virtual pseudo-filesystem that gives info about system and processes
/run	run-time variable data
/sys	virtial pseudo-filesystem that gives info about system and processes
/root	home directory of user
/sbin	essential system binaries
/srv	site-specific data
/tmp	temporary files
/usr	multi-user applications
/var	variable data that changes system operation

VPS (Virtual Private Server) that you can SSH into and work on. Reliable performance, data protection, control, etc.

Command line basics! This is where you tell the OS what to do. Remember, if you forget how a function works just do `man {function name}` or `--help`.

Moving: `cd` allows you to move between directories, `pwd` allows you to see your location, and `ls` lets you see the contents in the folder you're in.

Manipulating Files: `touch {file name}` lets you make a file, while `nano {file name}` lets you make a file and opens up an editor. `mkdir {new folder name}` makes a new directory while `rmdir` removes it. `cp` copies files, `mv` moves them, and `rm` removes them.

Viewing: `cat {file name}` prints all the contents in the file. `head` shows you the default top 10 lines, and you can add `head -n {number}` to see more or less specific. `tail` works the same, but shows you the ending.

Text Processing: `echo "message" > {file name}` allows you to print into something or simply onto the screen. `grep "find message" {file name}` returns every line that includes the message. `sed 's/{text to replace}/{replacement text}' {file name}` will replace all things from one message with another given a file name. To save the changes, add the tag `-i`. `sort {file name}` lexicographically sorts the file name from smallest to largest, and the extra argument after `| uniq` only shows distinct numbers.

Boot Sequence steps:

- BIOS(Basic I/O System)/UEFI locates and executes the boot program or boot loader
- Boot loader loads the kernel
- Kernel starts the init process (pid=1)
- Init manages system initialization, use `systemd`

BIOS contains all the code required to gain initial access to the keyboard, display screen, disk-drives, serial communications, etc. Typically placed in ROM chip (called ROM BIOS).

Almost all Linux distros use GRUB (GRand Unified Bootloader). GRUB can boot multiple OSs, network-based diskless booting, etc.

System-wide config files are generally placed in `/etc` and their subdirectories, while ultra-specific are in `home`. Only text files should be in `/etc`. Files in `/etc/sysconfig` and its subdirectories are used by many system utilities services.

`shutdown` is used to bring the system down in a secure fashion.

Week 3: Scripting + Bash

Scripting is designed to automate tasks in the Linux interface. A Shell script is essentially a text file that contains a sequence of commands for a UNIX based OS. A Python script is a plain text file with the python extension containing Python text to execute with an interpreter.

A shell is a program that takes commands from user and gives it to the OS to process, an example of CLI (Command-Line Interface). Examples include, **bash**, **zsh**, **ksh**, **tcsh**. A terminal is a program that allows you to interact with the shell.

This is what the **date** command does.

```
$ date
Thu Jul 11 14:42:19 EST 2026
```

You will always need to start your bash script, ending in extension **.sh**, with this:

```
#!/bin/bash
```

Here's an example where we can store a variable and also use the **echo** command:

```
#!/bin/bash

echo "What is your name?"
read PERSON
echo "Hello, $PERSON"
```

```
# in terminal
$ ./test.sh
What is your name?
Creed!
Hello, Creed!
```

To actually make the bash script executable, we use the following command: **chmod +x test.sh**.

Week 4: Services and Databases

2 main databases + use cases:

- Relational databases: data stored in multiple, related tables. Adhere to ACID (Atomicity, Consistency, Isolation, and Durability). Usually structure data with predefined, fixed relations EXAMPLE: Oracle, MySQL, PostgreSQL
- NoSQL (Non-Relational): data does not conform to predefined schema. Store unstructured data, or semi-structured. EXAMPLE: MongoDB

4 types of NoSQL databases:

- Doc-oriented: main unit is a document which is JSON containing different fields. Good use case would be good for data analysis and looking through records
- Columnar: main unit is a column of a table, retrieved column wise. Log-base files good use case.
- Key-value: all data has a unique key identifier. Good for dealing with cache

Relational Operators

-eq or = (string operators)	checks if value of two operands is equal or not	[\$a -eq \$b] is not true
-ne or != (string operators)	checks if value of two operands are not true	[\$a -ne \$b] is true
-gt	checks if the value of the left operand is greater than the value of the right operand	[\$a -gt \$b] is not true
-lt	checks if the value of the left operand is less than the value of the right operand	[\$a -lt \$b] is true
-ge	checks if the value of the left operand is greater than or equal to the value of the right operand	[\$a -ge \$b] is not true
-le	checks if the value of the left operand is less than or equal to the value of the right operand	[\$a -le \$b] is true
!	logical negation	[!false] is true
-o	logical OR	[1 -o 0] is true
-a	logical AND	[1 -a 0] is false
-z	checks if given string operand size is zero	[-z \$a] is not true
-n	checks if given string operand size is non-zero	[-n \$a] is not false
str	checks if str is not the empty string	[\$a] is not false

Unix File Test Operators

-b file	Checks if the file is a block special file; if yes, then the condition becomes true.	[-b \$file] is false.
-c file	Checks if file is a character special file; if yes, then the condition becomes true.	[-c \$file] is false.
-d file	Checks if the file is a directory; if yes, then the condition becomes true.	[-d \$file] is not true.
-f file	Checks if file is an ordinary file as opposed to a directory or special file; if yes, then the condition becomes true.	[-f \$file] is true.
-g file	Checks if the file has its set group ID (SGID) bit set; if yes, then the condition becomes true.	[-g \$file] is false.
-k file	Checks if the file has its sticky bit set; if yes, then the condition becomes true.	[-k \$file] is false.
-p file	Checks if the file is a named pipe; if yes, then the condition becomes true.	[-p \$file] is false.
-t file	Checks if the file descriptor is open and associated with a terminal; if yes, then the condition becomes true.	[-t \$file] is false.
-u file	Checks if the file has its Set User ID (SUID) bit set; if yes, then the condition becomes true.	[-u \$file] is false.
-r file	Checks if the file is readable; if yes, then the condition becomes true.	[-r \$file] is true.
-w file	Checks if the file is writable; if yes, then the condition becomes true.	[-w \$file] is true.
-x file	Checks if the file is executable; if yes, then the condition becomes true.	[-x \$file] is true.
-s file	Checks if the file has size greater than 0; if yes, then the condition becomes true.	[-s \$file] is true.
-e file	Checks if file exists; is true even if file is a directory but exists.	[-e \$file] is true.

- Graph: main unit contains nodes that represent entities and edges that represent relationships. Good for graph like structures and social networks.

Database management system (DBMS)

SQL (Structured Query Language) used on Relational DBMS

Cardinality - distinctiveness of info values contained in a column. High cardinality means column contains an outsized proportion of distinctive values. Low cardinality means more repeats. Mapping constraint is a data constraint that expresses the # of entities to which another entity can be related via a relationship set. For binary relationship set R on an entity A and B, there are four possible mapping cardinalities:

- One to one 1:1
- One to many 1:M
- Many to one M:1
- Many to Many M:M

Service - group of processes (known as daemons) running continuously in background. Most Linux distros use same process manager: systemd

Systemd is a system + service manager for Linux, allows parallel startup of system services at boot time, on-demand activation of daemons, and dependency based on service control logic. **.service** -> system services, **.target** -> group of systemd units. Main features:

- Socket-based activation allows parallel services to start without losing messages when restarted.
- Starts services in parallel as soon as all listening sockets are in place. Significantly reduces time required to boot system
- Only stops running services on shutdown

systemctl - command used to control and manage **systemd** services

Activate - **systemctl start foo**, deactivate - **systemctl stop foo**, restart - **systemctl restart foo**, view status - **systemctl status foo**. More commands -> **systemctl status foo**

HTTP (hypertext transfer protocol) - set of rules for sending and receiving web pages

Web request - communicative message that goes between client/web browser to servers.

HTTP Communication happens between client and server. Client makes HTTP request and gets back a HTTP response.

HTTP request line consists of Method, URL and Version

METHODS:

- GET: used to seek info from specified source.
- HEAD: only returns headers

- POST: seeks to create and make a new object
- PUT: replace fully with new section
URL: location that HTTP is referring to
VERSION: what version? HTTP/3? HTTP/2? HTTP/1.1?
HTTP request is followed by HTTP Header Structure:
- General: applied on request and response w/o affecting database body
- Request: contains info about fetched
- Response: contains location of source requested
- Entity: info about the body of resources

HTTP response has initial status line, some header lines and an entity body
Status Codes:

- 1xx: informational
- 2xx: success (200 is request was successful)
- 3xx: redirection
- 4xx: client error (404 is file not found, 400 is bad request/not in format system can read)
- 5xx: server error (500 is server error, 505 is requested HTTP version not supported)

Command: `curl http://example.org {head -silent}`: curl is name of command, `-head` or `-I` tells curl to send an HTTP request with head method. And `-silent` tells curl to not display progress.

Caching - supplements primary database by removing unnecessary pressure on it. Different types of caching:

- Database Integrated: managed within database engine and has built in write through capabilities. The cons are size and capabilities.
- Local: stored within application, removing network traffic. Con is that each app has its own node, thus it cannot shared with other local caches.
- Remote: stored in dedicated servers by NoSQL.

Week 5: Testing

We need to make sure our code is always working as expected. Testing is conducted to ensure systems and applications are robust to prevent issues. We also test deployment so they must make sure CI/CD(Continuous integration/delivery/deployment) pipelines are ensured.

Types of Testing:

- White Box Testing: internal structure, design and coding of software are tested to verify flow of I/O and to improve design, usability, and security. Code is visible to testers.
- Black Box Testing: functionalities of software applications are tested without having knowledge of internal code structure, implementation details, and internal paths.
- Gray Box Testing: tests software product or application with partial knowledge of the internal structure of application.

Testing Levels:

- Unit Testing: tests individual components of the software. Makes sure all the individual parts work as intended. It is ideal to do this test prior to Integration Testing. Advantages include: quality of code improved, able to find bugs easier, changing code becomes easier, easier to understand project through documentation, debugging process simplified.

```
# if this is code to calculate volume of Cube:
def cuboid_volume(l):
    return (l*l*l)

# then let's write a Unit Test!
import unittest
class TestCuboid(unittest.TestCase):
    def test_volume(self):
        self.assertEqual(cuboid_volume(2),8)
        self.assertEqual(cuboid_volume(1),1)
        self.assertEqual(cuboid_volume(0),0)
        self.assertEqual(cuboid_volume(5.5),166.375)
```

- Integration Testing: tests the combination of different units, their interactions, subsystem unity, and code compliance with the requirements. Ideal for after Unit Tests. Advantages include: simultaneous testing of different modules becomes convenient, lifesaver for a separated team, covers large amount of code. To implement it, use **pytest** library.
- System Testing: tests the system as a whole. All the modules/components are integrated in order to verify the system works as expected or not. Ideal to do before giving it off as a final product or to go back to the drawing board. Advantages include: tests end-to-end scenarios to test the system. Tests the same environments as the production environment which understands perspective and prevents issues.

Always remember to start testing early! You should test a webpage like so:

- Functionality: Test all links, test forms, and cookies are working as expected and not broken.

- Usability: test the visibility and accessibility of the webpage and make sure the user can see and interact with everything easily. Navigation, content, etc.
- Interface: test to make sure the application is sending requests correctly to the database and outputting it right. Make sure the web server is handling the in between correctly and make sure all web queries appear in database.
- Database: test the database. Any errors in executing the queries, data integrity for creating, updating and deleting data, check response time, and test retrieval.
- Compatibility: make sure the webpage displays correctly across all devices and browsers.
- Performance: test response times and different connection speeds, load test the application and normal and peak loads, stress test to a breaking point and test a crash response.
- Security: test unauthorized access requests, accessing restricted files, check session that are killed for prolonged user activity, and use SSL certificates, websites should restrict to encrypted SSL pages.
- Crowd: get a large number of people(crowd) to execute tests.

There are also different types of tests to ensure all system, applications, environments and infrastructure are functioning as expected.

- Canary Releases: reduces risk of introducing new software version by slowly rolling out the change to a small subset of users first.
- Load Testing: simulates high demand on a software by testing to see max operating capacity of the app, whether current infrastructure is sufficient, is it sustainable, how many users can it support.
- Feature Flagging and Dark Launches: sees which features and functions can be revealed to the user or no. They leave the features disabled, and slowly enable them.
- Application Monitoring: actively monitoring application. RRUM(Real User Monitoring) where we track real world interaction. We can also use Synthetic monitoring where it reacts from simulated users.
- Tracing: we see a single interaction go through multiple layers and see if it follows the correct path.

Test Driven Development(TDD) focuses on creating unit test cases before developing the actual code. Once new code passes test, it is refactored to an acceptable standard. There are 3 phases in TDD:

- Create precise tests that verify functionality
- Correct the code once it fails
- Refactor the code once it passes to reduce redundancies or to optimize the code.

There are many benefits of TDD including creates more optimized code, helps developer better analyze and understand client requirements and requests, makes testing new functionalities easier, test coverage is higher to reduce later failure, and enhances overall productivity.

When we run the following script:

```
$ ls -l a\_file
-rw-rw-r-- 1 coop a\_project 1601 Mar 9 15:04 a\_file
```

The part, **rw-rw-r-**, meaning owner permissions(read and write), group permissions (read and write), and other permissions (read only). The user is **coop** and the group is **a_group**

There are several types of permissions that can be given: **r** is read access, **w** is write access, and **x** is execute access. If not given, the placement is replaced by a dash. Other specialized permissions exist. **setuid** allows an executable to run with owner privileges, while **setgid** runs on group privileges.

If we want to change file permissions, we use **chmod**. Here's an example where we allows owner and world to have execute privileges and remove the group write permissions:

```
$ ls -l a\_file
-rw-rw-r-- 1 coop coop 1601 Mar 9 15:04 a\_file
$ chmod uo+x, g-w a\_file
$ ls -l a\_file
-rwxr--r-x 1 coop coop 1601 Mar 9 15:04 a\_file
```

The **u** stands for user(owner), **o** stands for other(world), **g** stands for group. You can also use octal digits for shorthand chmodding, using 3 digits for each person to give permission (user, group, world).

- 1 - if the execute permission is desired
- 2 - if the write permission is desired
- 4 - if the read permission is desired
- 5 - means read/execute
- 6 - means read/write
- 7 - means read/write/execute

Whenever we need to change file ownership, we use **sudo chown [new user] [file]** and to change group ownership we use **sudo chgrp [new group] [file]**. Only superuser can do this.

POSIX ACLs(Access Control Lists) extend the simple user, group, and world system.

Groups are defined in **/etc/group** where every line looks like this: **groupname:password:GID:user1,user2...**

- Groupname is the name of the group
- password is the password placeholder, only may be set if **/etc/gshadow** exists
- GID is the group identifier. Values 0-99 are for system groups. Values between 100-GID_MIN are considered special. Values above GID_MIN are for UPG(User Private Groups).
- Then finally a list of all the members of the group.

The following may allow you to modify group settings:

- **groupadd** - add a new group
- **groupmod** - modify a group and add new users
- **groupdel** - remove a group
- **usermod** - manage a user's group memberships

HackerRank Questions

Reverse a Linked List:

"""

Complete the 'reverse' function below.

The function is expected to return an INTEGER_SINGLY_LINKED_LIST.

The function accepts INTEGER_SINGLY_LINKED_LIST llist as parameter.

For your reference:

SinglyLinkedListNode:

int data

SinglyLinkedListNode next

"""

```
def reverse(llist):
    # Write your code here
    prev = None
    current = llist

    while current is not None:
        next_node = current.next
        current.next = prev
        prev = current
        current = next_node

    return prev

if __name__ == '__main__':
    fptr = open(os.environ['OUTPUT_PATH'], 'w')

    tests = int(input())

    for tests_itr in range(tests):
        llist_count = int(input())

        llist = SinglyLinkedList()

        for _ in range(llist_count):
            llist_item = int(input())
            llist.insert_node(llist_item)

        llist1 = reverse(llist.head)

        print_singly_linked_list(llist1, ' ', fptr)
```

```

        fptr.write( '\n' )

    fptr.close()

Cycle Detection:
"""
Complete the has_cycle function below.
For your reference:

SinglyLinkedListNode:
    int data
    SinglyLinkedListNode next
"""

def has_cycle(head):
    if head is None:
        return False

    slow = head
    fast = head

    while fast is not None and fast.next is not None:
        slow = slow.next
        fast = fast.next.next

        if slow == fast:
            return True

    return False

if __name__ == '__main__':

```